تابع المحاضرة (1)
تصميم برمجيات

مبادئ التصميم الجيد:

مبادئ التصميم الجيد للبرمجيات تشمل مجموعة من القواعد والتوجيهات التي تساعد في إنشاء برمجيات فعالة وقابلة للصيانة.

1. **البساطة (Simplicity)**
   - جعل التصميم بسيطًا قدر الإمكان. كلما كان التصميم أقل تعقيدًا، كان أسهل في الفهم والصيانة.

2. **الوضوح (Clarity)**
   - يجب أن يكون الكود واضحًا وسهل القراءة. استخدام أسماء متغيرات ووظائف معبرة يساعد في فهم الكود بسرعة.

3. **التحكم في التغيرات (Change Control)**
   - صمم النظام بحيث يسهل إجراء تغييرات عليه دون التأثير على الأجزاء الأخرى. هذا يعزز من القابلية للصيانة.

4. **إعادة الاستخدام (Reusability)**
   - استخدم مكونات يمكن إعادة استخدامها في مشاريع أخرى. هذا يقلل من الجهد المطلوب لتطوير ميزات جديدة.
5. **فصل المهام (Separation of Concerns)**
   - قسم النظام إلى مكونات مستقلة تعالج جوانب مختلفة من التطبيق، مما يسهل فهم كل جزء على حدة.

6. **التغليف (Encapsulation)**
   - جعل البيانات والوظائف ذات الصلة في كائنات أو وحدات مغلقة، مما يمنع الوصول غير المصرح به ويقلل من الاعتماد على تفاصيل التنفيذ.

7. **التوسع (Scalability)**
   - صمم النظام بحيث يمكنه التوسع بسهولة لاستيعاب زيادة في حجم البيانات أو عدد المستخدمين دون التأثير على الأداء.
8. **الاختبار (Testability)**
   - اجعل التصميم قابلاً للاختبار بشكل سهل. يجب أن تكون وحدات الكود قابلة للاختبار بشكل مستقل.

9. **التوافق (Consistency)**
   - يشير التوافق إلى الاتساق والتناسق في تصميم البرمجيات، ويعني أن الأجزاء المختلفة من النظام تعمل بشكل متناغم مع بعضها البعض.

10. **التوثيق (Documentation)**
   - قم بتوثيق التصميم والكود بشكل جيد، مما يسهل على المطورين الآخرين فهم النظام وصيانته.

إعادة الاستخدام في تصميم البرمجيات

بعض الجوانب الرئيسية لإعادة الاستخدام:

1. **المكونات القابلة لإعادة الاستخدام:**
   - تشمل الفئات(classes) ، المكتبات(libraries) ، والوحدات (modules) التي يمكن استدعاؤها في مشاريع مختلفة دون الحاجة إلى إعادة كتابتها.
2. **الكفاءة:**
   - يعزز إعادة الاستخدام من كفاءة تطوير البرمجيات، حيث يمكن للمطورين الاستفادة من الكود الموجود بدلاً من البدء من الصفر.
3. **تقليل الأخطاء:**
   - استخدام مكونات تم اختبارها مسبقًا يقلل من فرص الأخطاء، حيث تكون هذه المكونات قد تم تحسينها وإصلاحها في سياقات سابقة.
4. **تحسين الصيانة:**
   - إذا تم تعديل مكون واحد، فإن جميع التطبيقات التي تستخدمه يمكن أن تستفيد من هذا التعديل، مما يسهل الصيانة.
5. **الهندسة المعمارية:**
   - يمكن أن تسهم إعادة الاستخدام في بناء أنظمة معمارية أكثر مرونة، مثل استخدام نمط البرمجة الكائنية (OOP) أو خدمات الويب.(Web Services)
6. **المكتبات والأطر:**
   - استخدام مكتبات وأطر عمل (frameworks) معروفة يمكن أن يوفر الوقت ويزيد من الإنتاجية، حيث تحتوي على مجموعة من الوحدات القابلة لإعادة الاستخدام.

الفوائد:

- **توفير الوقت :**يقلل من الوقت المستغرق في تطوير البرمجيات الجديدة.
- **تحسين الجودة :**يعزز من جودة البرمجيات من خلال استخدام مكونات مثبتة وموثوقة.
- **تعزيز الابتكار :**يسمح للمطورين بالتركيز على تطوير ميزات جديدة بدلاً من إعادة بناء نفس المكونات.

التوافق في مبادئ التصميم الجيد

بعض الجوانب الرئيسية للتوافق:

1. **التصميم المتسق:**
   - يجب أن تتبع جميع مكونات التطبيق أنماط تصميم متشابهة، مما يسهل على المستخدمين فهم كيفية التفاعل مع النظام. على سبيل المثال، استخدام ألوان وأزرار وخطوط موحدة.
2. **النمط البرمجي:**
   - يجب أن تكون الأساليب والتقنيات المستخدمة في كتابة الكود متسقة عبر المشروع. هذا يساعد المطورين على فهم الكود بشكل أسرع ويقلل من الأخطاء.
3. **تجربة المستخدم:**
   - يجب أن تكون تجربة المستخدم متسقة عبر جميع أجزاء التطبيق. على سبيل المثال، إذا كان هناك نموذج إدخال بيانات في جزء من التطبيق، يجب أن يكون له نفس الشكل والوظيفة في الأجزاء الأخرى.
4. **التوافق مع الأنظمة الأخرى:**
   - يجب أن تكون البرمجيات قادرة على التفاعل بشكل جيد مع أنظمة أو تطبيقات أخرى، مما يسهل تكاملها في بيئات متعددة.
5. **التوافق مع المعايير:**
   - يجب أن تتوافق البرمجيات مع المعايير الصناعية أو المتطلبات القانونية، مما يسهل قبولها واستخدامها في مجالات معينة.

الفوائد:

- **سهولة الاستخدام :**يجعل النظام أكثر سهولة في التعلم والاستخدام.
- **تقليل الأخطاء :**يقلل من فرص حدوث الأخطاء الناتجة عن عدم التناسق.
- **الكفاءة :**يسهل عمليات الصيانة والتحديث، حيث يعرف المطورون كيفية العمل مع النظام بشكل أسرع.